

Object-Oriented Programming (OOP) in Python

A small quick-reference guide

1. What is OOP?

Object-Oriented Programming is a programming approach that organizes code using **classes** and **objects**. It helps combine data and the functions that operate on that data into reusable units.

2. Class and Object

A **class** is a blueprint for creating objects. An **object** is an instance of a class.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car1 = Car("Toyota")
print(car1.brand)
```

3. Constructor: __init__()

The **__init__()** method is a special method that is automatically called when an object is created. It is commonly used to initialize object attributes.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

4. The self Keyword

self refers to the current object. It is used to access attributes and methods belonging to that object.

```
class Person:
    def greet(self):
        print("Hello")

p = Person()
p.greet()
```

5. Encapsulation

Encapsulation means keeping data and related methods together and controlling access to internal data. Python commonly uses naming conventions such as a single underscore (**_**) and name mangling with double underscores (**__**).

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        self.__balance += amount
```

6. Inheritance

Inheritance allows a child class to reuse attributes and methods from a parent class.

```
class Vehicle:
    def move(self):
```

```
print("Moving")

class Car(Vehicle):
    pass

c = Car()
c.move()
```

7. Polymorphism

Polymorphism allows the same method name or interface to behave differently for different objects.

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")
```

8. Abstraction

Abstraction hides implementation details and exposes only the essential interface. Python provides the **abc** module for defining abstract base classes.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

9. Four Main OOP Concepts

Concept	Meaning
Encapsulation	Bundle data and methods; control access
Inheritance	Reuse features from an existing class
Polymorphism	Same interface, different behavior
Abstraction	Hide implementation details

10. Important Points to Remember

- Class = blueprint; Object = instance of a class.
- `__init__()` is commonly used as a constructor/initializer.
- `self` refers to the current object.
- Inheritance promotes code reuse.
- Polymorphism allows different implementations through a common interface.
- Abstraction focuses on what an object does rather than how it does it.